Universidad del Valle de Guatemala

Construcción de Compiladores · Sección 10

COMPISCRIPT STUDIO

ANTLR · TAC · MIPS32 · MARS

Informe técnico

Proyecto 3 · Generación de código ensamblador

Grupo: Daniel-Barillas
Integrante: Pablo Daniel Barillas Moreno
Carné: 22193
Versión: 3.0.0
Fecha: 28 de octubre de 2026

Compiscript → TAC → MIPS32 → MARS 4.5

Índice general

1. Resumen ejecutivo **1**

2. Objetivos, alcance y criterios de aceptación **2**

2.1. Problema que resuelve el proyecto . . . 2
2.2. Objetivo general . . . 2
2.3. Objetivos específicos . . . 2
2.4. Alcance funcional . . . 3
2.5. Restricciones obligatorias y respuesta de diseño . . . 3
2.6. Criterios de aceptación . . . 3
2.7. Tecnologías empleadas . . . 4

3. Fundamentos teóricos **5**

3.1. Fases de compilación . . . 5
3.2. Análisis léxico y sintáctico . . . 5
3.3. Análisis semántico . . . 6
3.4. Código de tres direcciones . . . 6
3.5. Arquitectura MIPS32 y MARS . . . 7
3.6. Registros de activación . . . 7
3.7. Heap, referencias y tiempo de vida . . . 8

4. Arquitectura de la solución **9**

4.1. Organización por capas . . . 9
4.2. Contratos y estados entre fases . . . 9
4.3. Identidad de símbolos y trazabilidad . . . 10
4.4. Determinismo y reproducibilidad . . . 10

5. Diseño e implementación **11**

5.1. Contrato del backend . . . 11
5.2. Proceso de generación . . . 11
5.3. Resolución de almacenamiento . . . 11
5.4. Asignador getReg . . . 12

5.5. Selección de instrucciones . . . 12
5.6. Convención de llamadas paso a paso . . . 13
5.7. Arreglos y matrices . . . 13
5.8. Objetos, layouts y despacho dinámico . . . 13
5.9. Closures y capturas . . . 14
5.10. Strings y flotantes . . . 14
5.11. Errores de runtime y try/catch . . . 14
5.12. Mirilla . . . 14
5.13. Validación del ensamblador generado . . . 15

6. Ejemplo completo de traducción 16
6.1. Programa fuente . . . 16
6.2. Representación intermedia . . . 16
6.3. Decisiones de almacenamiento . . . 17
6.4. Esqueleto MIPS resultante . . . 17
6.5. Qué ocurre si el programa es inválido . . . 17
6.6. Lectura comparativa del ejemplo . . . 18

7. Interfaz y experiencia de uso 19
7.1. Distribución del workbench . . . 19
7.2. Flujo de uso . . . 19
7.3. Exportación de evidencia . . . 20

8. CLI y ejecución externa 21
8.1. Modos de la CLI . . . 21
8.2. Ejecución reproducible en PowerShell . . . 21
8.3. Diagnóstico de la ejecución externa . . . 22

9. Pruebas y resultados 23
9.1. Estrategia de verificación . . . 23
9.2. Suite automatizada . . . 23
9.3. Validación real en MARS . . . 24
9.4. Pruebas negativas y recuperación . . . 24
9.5. Comandos de verificación . . . 25
9.6. Interpretación de resultados . . . 25
9.7. Integración continua . . . 25

10. Trazabilidad con el enunciado **27**
10.1. Evidencia observable durante la demostración 27
10.2. Responsabilidad individual 27

11. Limitaciones y trabajo futuro **28**

12. Conclusiones **29**

A. Mapa de archivos **30**

B. Catálogo resumido de diagnósticos **31**

C. Comandos de reproducción **32**
C.1. Preparación y desarrollo 32
C.2. Verificación completa 32
C.3. Generación y ejecución manual 32

CAPÍTULO 1

Resumen ejecutivo

El Proyecto 3 completa un compilador educativo para Compiscript. Conserva el lexer y parser generados con ANTLR4TS, el análisis semántico con tabla de símbolos y la representación intermedia de tres direcciones de las entregas anteriores. Sobre esa base incorpora un backend que transforma exclusivamente TAC en ensamblador MIPS32 compatible con MARS 4.5.

El producto se presenta como una interfaz gráfica React con editor Monaco. El usuario puede escribir o seleccionar un archivo `.cps`, ejecutar el pipeline, observar diagnósticos y examinar tokens, CST, árbol semántico, símbolos, ámbitos, TAC, bloques básicos, registros de activación y ensamblador. La exportación MIPS se habilita solamente cuando todas las fases terminan correctamente.

El backend implementa convención de llamadas mediante pila, asignación local de registros, aritmética entera y flotante, control de flujo, funciones, llamadas anidadas, recursión, arreglos, objetos, métodos con despacho dinámico, strings, closures y recuperación de errores de ejecución mediante `try/catch`. La suite define 41 casos de ejecución MARS y el workflow los exige con Java 17.

El resultado no es solamente un traductor de ejemplos aislados. Es un pipeline verificable con contratos entre fases, bloqueo seguro ante entradas inválidas, diagnósticos acumulativos y evidencia reproducible. La versión documentada contiene 232 pruebas automatizadas en 15 archivos: 191 verificaciones que no requieren herramientas externas y 41 ejecuciones reales del ensamblador generado en MARS 4.5. El informe explica las decisiones que permiten sostener esa compatibilidad, los riesgos que se corrigieron y la relación entre cada requisito y su evidencia.

> **Resultado principal.** Un archivo Compiscript válido puede seleccionarse en la interfaz, recorrer lexer, parser, semántica y TAC, producir MIPS32, descargarse como `.asm` y ejecutarse externamente en MARS. Si aparece un error léxico, sintáctico o semántico, las fases TAC y MIPS quedan omitidas y no se habilita una exportación potencialmente incorrecta.

CAPÍTULO 2

Objetivos, alcance y criterios de aceptación

2.1. Problema que resuelve el proyecto

El punto de partida es un lenguaje de alto nivel con variables, tipos, estructuras de control, funciones, arreglos, clases, herencia y funciones anidadas. La máquina destino, en cambio, opera con registros, palabras de memoria, saltos y direcciones. El reto del Proyecto 3 consiste en cerrar esa diferencia sin perder el significado validado en las fases anteriores.

La transformación debe conservar cuatro propiedades. Primero, cada identificador debe referirse al símbolo correcto, incluso con *shadowing*. Segundo, cada llamada debe preservar el estado del llamador y permitir recursión. Tercero, los datos dinámicos deben tener una representación concreta y segura en heap. Cuarto, la salida debe ser aceptada por una herramienta externa real, no solamente parecer ensamblador.

2.2. Objetivo general

Diseñar e implementar un backend que reciba exclusivamente la representación de tres direcciones de Compiscript y genere ensamblador MIPS32 ejecutable en MARS 4.5, integrado a una interfaz gráfica que permita observar y exportar los resultados del proceso de compilación.

2.3. Objetivos específicos

1. Reutilizar el lexer y parser generados con ANTLR4TS, el analizador semántico y el TAC, sin introducir un segundo frontend incompatible.
2. Definir una convención de llamadas que soporte parámetros, retornos, llamadas anidadas, métodos y recursión.
3. Traducir tipos primitivos y compuestos a una representación uniforme de palabras de 32 bits y bloques de heap.
4. Implementar comprobaciones de runtime para accesos de arreglo, referencias nulas y divisiones entre cero.
5. Aplicar optimizaciones de mirilla conservadoras y medir su efecto sin alterar la semántica.
6. Exponer diagnósticos, TAC, frames y ensamblador dentro de una GUI, con selección de archivos y descarga controlada.
7. Demostrar el resultado con pruebas unitarias, pruebas de integración, ejecuciones MARS y un workflow reproducible.

2.4. Alcance funcional

La entrega incluye el flujo completo desde texto fuente hasta un archivo ensamblador. El lenguaje cubierto contiene declaraciones `let`/`const`, expresiones, booleanos, enteros, flotantes, strings, arreglos multidimensionales, condicionales, ciclos, `switch`, funciones, recursión, closures, clases, constructores, herencia, sobrescritura y `try/catch`. El backend genera segmentos `.data` y `.text`, runtime auxiliar y metadatos de validación.

Quedan fuera del alcance la generación de código objeto, el enlazado nativo, un recolector de basura y una fase global de optimización. La ejecución del MIPS tampoco ocurre dentro de la GUI: el producto genera y exporta, mientras MARS ensambla y ejecuta como herramienta externa. Esta separación coincide con el enunciado y mantiene claro el límite de responsabilidad del compilador.

2.5. Restricciones obligatorias y respuesta de diseño

Restricción	Respuesta implementada
Generador obligatorio de analizadores	ANTLR4TS genera lexer, parser y Visitor desde `Compiscript.g4`; no se sustituyó por un parser manual.
Ensamblador obligatorio desde TAC	La API del backend recibe `TacGenerationResult`; no consulta el CST para reconstruir el programa.
Interfaz gráfica	Workbench React con Monaco, explorador, pipeline, problemas, inspectores y temas claro/oscuro.
Selección de archivo en GUI	El menú Archivo permite cargar entradas `.cps` sin depender de la consola.
Resultados visibles en GUI	Tokens, àrboles, símbolos, àmbitos, TAC, frames, MIPS y métricas aparecen en paneles internos.
Recuperación de errores	Listeners, estrategia de ANTLR, normalización y deduplicación permiten reportar varios problemas por ejecución.
No generar tras errores	Un error previo establece TAC/MIPS en `skipped`; la descarga se habilita solo para `completed`.
Optimización de mirilla	Cuatro reglas locales eliminan instrucciones redundantes y reportan conteos antes/después.
Ejecución externa	El `.asm` se valida con MARS 4.5 mediante Java y el runner incluido.

2.6. Criterios de aceptación

Se considera que una compilación válida concluye correctamente cuando no hay errores, TAC y MIPS terminan en `completed`, el ensamblador contiene entrada, datos y etiquetas resueltas, y MARS produce la salida esperada. Una compilación inválida es correcta cuando acumula diagnósticos ùtiles, no cae

en un ciclo de recuperación, omite TAC/MIPS y deshabilita la exportación. Estos dos caminos se prueban explícitamente; no se interpreta la ausencia de una excepción de JavaScript como evidencia suficiente.

2.7. Tecnologías empleadas

Capa	Tecnología	Propósito
Frontend	ANTLR4TS	Generación de lexer, parser y Visitor
Implementación	TypeScript	Tipado de contratos, semántica, TAC y backend
Interfaz	React 19, Monaco, Tailwind CSS, Motion, Radix UI	Experiencia de IDE, paneles y componentes accesibles
Construcción	Vite 6	Desarrollo y empaquetado web
Pruebas	Vitest 5	Pruebas unitarias, de regresión e integración
Destino	MIPS32 / MARS 4.5	Ensamblado y ejecución externa verificable
Escritorio	Electron	Distribución opcional para Windows, macOS y Linux
Automatización	GitHub Actions	Verificación con Java, MARS, build y empaquetado

CAPÍTULO 3

Fundamentos teóricos

3.1. Fases de compilación

Un compilador transforma un lenguaje fuente en una representación destino mediante fases con contratos definidos. En este proyecto, los tokens del lexer alimentan al parser; el CST válido alimenta al visitor semántico; el resultado tipado habilita TAC; finalmente, TAC habilita MIPS.

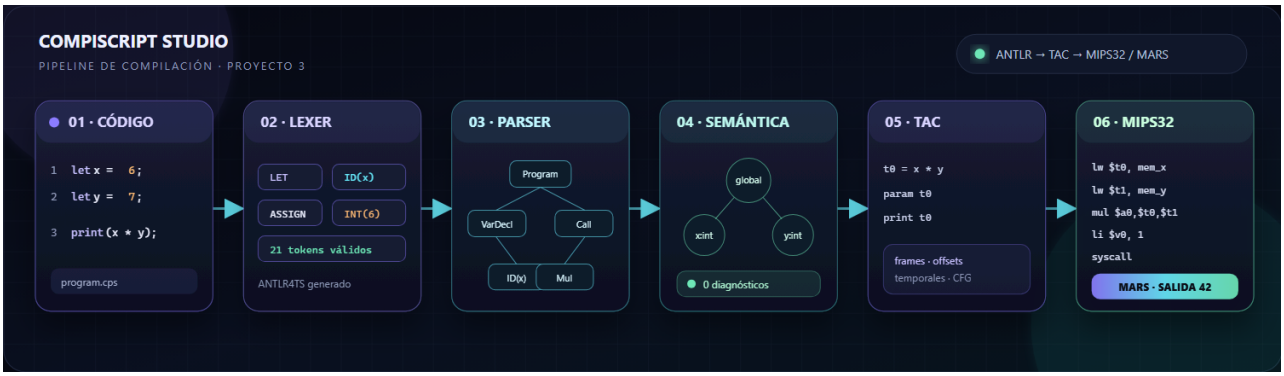


Figura 3.1: Pipeline completo de la versión 3.0.0.

El principio de bloqueo evita que un error temprano se convierta en muchos errores derivados: lexer y parser continúan recolectando diagnósticos, pero TAC y MIPS no se ejecutan si el programa no posee significado válido.

3.2. Análisis léxico y sintáctico

El lexer convierte caracteres en tokens: identificadores, literales, operadores, palabras reservadas y delimitadores. Los caracteres inválidos no detienen inmediatamente el proceso; se registran con línea y columna y el flujo continúa. Para evitar ruido, caracteres inválidos contiguos y errores derivados se normalizan antes de mostrarse.

El parser consume esos tokens y construye un *concrete syntax tree* (CST) conforme a la gramática. La estrategia de recuperación intenta sincronizarse en puntos seguros, como cierres de instrucción o bloque, para alcanzar declaraciones posteriores. Los mensajes internos de ANTLR se traducen a descripciones orientadas al usuario. Además, un límite estructural de 256 niveles rechaza entradas patológicamente anidadas antes de que la pila de JavaScript se agote.

La recuperación tiene una consecuencia importante: permite informar varios problemas en una sola ejecución, pero no autoriza a continuar con fases que asumirían un àrbol correcto. Por eso el CST puede mostrarse para diagnóstico, mientras la semántica, TAC y MIPS respetan el estado de las fases anteriores.

3.3. Análisis semántico

La sintaxis confirma que una construcción tiene forma válida; la semántica determina si tiene significado. Un programa puede estar bien formado y aun así intentar sumar una función, usar una variable no declarada, llamar un constructor con argumentos incompatibles o retornar un valor del tipo equivocado.

El analizador mantiene una jerarquía de àmbitos: global, clase, función, bloque, ciclo, `switch` y `catch`. Cada entrada de la tabla de símbolos posee un identificador estable, tipo, mutabilidad, ubicación y almacenamiento. Buscar por identidad, y no solamente por nombre, es esencial para distinguir dos variables llamadas igual en bloques anidados.

Categoría	Ejemplos	Regla principal
Primitivos	integer, float, boolean, string, null	operaciones y conversiones según tipo
Arreglos	`T[]`, matrices anidadas	elementos homogéneos y asignabilidad invariante
Instancias	clases y subclases	una instancia hija es asignable a una referencia base
Funciones	parámetros y retorno	aridad y tipos compatibles
Control	if, ciclos, switch, return	condiciones booleanas y contexto válido
Miembros	this, campos y métodos	resolución en la clase y su cadena de herencia

La promoción implícita admitida es `integer` hacia `float`. Los arreglos mutables son invariantes: aunque `Perro` sea subtipo de `Animal`, `Perro[]` no se trata como `Animal[]`, porque a través de esa referencia podría insertarse otra subclase y romper el tipo real del arreglo.

3.4. Código de tres direcciones

TAC descompone expresiones complejas en instrucciones con a lo sumo dos operandos fuente y un resultado. Por ejemplo, `x = a + b * 4` se convierte en:

```
t0 = b * 4
t1 = a + t0
x = t1
```

Esta forma simplifica la selección de instrucciones: cada cuádruplo tiene una traducción local y los metadatos de tipo determinan si debe usarse la ALU entera o el coprocesador flotante.

Cada instrucción TAC incluye `op`, hasta dos argumentos, resultado, àmbito, frame y ubicación de origen. Esta información evita que el backend tenga que volver a inferir decisiones ya resueltas. También permite exportar evidencia en CSV y relacionar una instrucción con el programa fuente.

Familia TAC	Operaciones representativas	Responsabilidad
Datos	MOV, PARAM, RETURN, PRINT	mover, pasar, retornar y observar valores
Aritmética	ADD, SUB, MUL, DIV, MOD, NEG	cálculo entero o flotante
Control	LABEL, GOTO, IF_TRUE, IF_FALSE	construir el grafo de flujo
Funciones	FUNC_BEGIN, FUNC_END, CALL	delimitar frames y llamadas
Arreglos	NEW_ARRAY, ARRAY_GET, ARRAY_SET	heap, longitud e indexación segura
Objetos	NEW_OBJECT, GET_FIELD, SET_FIELD	instancias, layouts y campos
Closures	MAKE_CLOSURE, CAPTURE, LOAD_CAPTURE	funciones como valores y entornos
Excepciones	TRY_BEGIN, TRY_END, CATCH_BEGIN	handlers dinámicos de runtime

Los saltos dividen TAC en bloques básicos. Una etiqueta inicia un bloque; un salto o retorno lo termina. El grafo de control permite inspeccionar sucesores, confirmar que los destinos existen y considerar una instrucción `RETURN` como terminadora, sin confundir el código posterior con una ruta ejecutable.

3.5. Arquitectura MIPS32 y MARS

MIPS emplea un conjunto reducido de instrucciones y registros explícitos. El proyecto usa registros temporales `$t0-$t9`, argumentos `$a0-$a1`, retornos `$v0`/`$f0`, stack pointer `$sp`, frame pointer `$fp`, return address `$ra` y el registro `$s7` como enlace al entorno de un closure.

MARS 4.5 ensambla MIPS32 y ofrece syscalls educativas. Se emplean syscall 1 para enteros, 2 para flotantes, 4 para strings, 9 para memoria dinámica y 10 para terminar el programa.

MIPS32 es apropiado para el proyecto porque hace visibles decisiones que una arquitectura más abstracta podría ocultar: dónde vive cada valor, quién preserva un registro, cómo se calcula un offset y cómo se retorna de una función. MARS agrega un entorno reproducible sin exigir hardware físico ni un sistema operativo MIPS.

3.6. Registros de activación

Cada llamada necesita un frame independiente para soportar recursión. El tamaño proviene del campo `totalBytes` del registro de activación y se alinea a ocho bytes.



Figura 3.2: Organización conceptual del frame.

El llamador apila argumentos de izquierda a derecha. La función reserva el frame, guarda estado y copia parámetros a offsets estables. Una `CALL` consume solo sus propios argumentos, lo cual resuelve correctamente llamadas anidadas.

Un registro de activación no es solo espacio temporal. Es la frontera entre una invocación y otra: dos llamadas recursivas a `factorial` comparten el mismo código, pero poseen parámetros, temporales y direcciones de retorno diferentes. El frame pointer ofrece una base estable aun cuando el stack pointer cambie al apilar argumentos de llamadas internas.

3.7. Heap, referencias y tiempo de vida

Los frames desaparecen al retornar; por eso no pueden almacenar datos que deban sobrevivir a una llamada. Arreglos, objetos, strings concatenados y entornos de closures se reservan en heap mediante syscall 9. Las variables conservan direcciones de 32 bits hacia esos bloques.

Esta separación explica el tratamiento especial de closures. Una variable capturada no puede quedarse exclusivamente en el frame de la función creadora, porque el closure podría ejecutarse después del retorno. El compilador la representa mediante una celda compartida en heap y guarda un puntero a esa celda en el descriptor del closure.

CAPÍTULO 4

Arquitectura de la solución

4.1. Organización por capas

La aplicación se divide en capas con responsabilidades observables. `src/lib/analyze.ts` orquesta el pipeline; `src/semantic` resuelve tipos, símbolos y flujo; `src/tac` construye la representación intermedia, frames y layouts; `src/mips` genera y valida ensamblador; `src/ui` presenta el resultado; y `src/cli` reutiliza el mismo orquestador desde la terminal.

Capa	Entrada / salida	Responsabilidad
Lexer	texto → tokens	reconocer unidades y acumular errores léxicos
Parser	tokens → CST	validar estructura y recuperarse en puntos seguros
Semántica	CST → modelo tipado	resolver símbolos, tipos, herencia y flujo
TAC	modelo válido → cuádruplos	normalizar operaciones y calcular metadatos
Backend	TAC → MIPS32	seleccionar instrucciones, runtime y mirilla
Presentación	resultado → paneles/archivos	inspeccionar, copiar y exportar evidencia

La CLI y la GUI no mantienen compiladores separados. Ambas invocan `analyzeInput`; por tanto, una corrección del frontend o backend beneficia los dos caminos y las pruebas no validan una implementación distinta de la presentada al usuario.

4.2. Contratos y estados entre fases

Cada fase recibe una estructura definida y produce estado, datos y diagnósticos. La siguiente fase solo se ejecuta si se cumplen sus precondiciones. Esta decisión evita dos fallos frecuentes: generar instrucciones a partir de un àrbol recuperado ambiguo y mostrar como exitoso un ensamblador parcial.

> **Regla de propagación:** un error léxico o sintáctico omite semántica, TAC y MIPS; un error semántico omite TAC y MIPS; una inconsistencia TAC omite MIPS; un diagnóstico del backend produce `failed`. Solamente `completed` habilita la descarga del ensamblador.

El estado `skipped` no equivale a un fallo interno: significa que la fase no debía ejecutarse debido a una precondición. `failed`, en cambio, indica que el backend fue solicitado pero detectó una referencia sin

resolver, una etiqueta faltante o una operación no traducible. La distinción mejora tanto la interfaz como la depuración.

4.3. Identidad de símbolos y trazabilidad

Los nombres son convenientes para el usuario, pero no son identidades globales. Dos declaraciones `valor` en àmbitos diferentes tienen `symbolId` distintos. TAC propaga esa identidad y el backend resuelve slots exclusivamente por ella. Esta regla corrige el riesgo de que una variable interior sobrescriba el almacenamiento de la exterior.

La ubicación fuente también viaja con las instrucciones. Cuando una fase reporta un error, puede apuntar a línea y columna; cuando se exporta TAC, se conservan esos campos. La trazabilidad permite explicar no solo qué ensamblador existe, sino qué decisión del programa lo originó.

4.4. Determinismo y reproducibilidad

Los contadores de etiquetas, temporales y diagnósticos se reinician entre compilaciones. Los temporales liberados pueden reutilizarse y las variables locales sin inicializador se escriben con cero en el prólogo. En consecuencia, la misma entrada produce una representación estable y no depende del contenido residual de la pila ni de una ejecución previa del IDE.

La reproducibilidad se extiende al entorno: `npm ci` usa el lockfile, `npm run generate` reconstruye los artefactos ANTLR, el setup de MARS verifica SHA-256 y CI fija Java 17 para las pruebas externas.

CAPÍTULO 5

Diseño e implementación

5.1. Contrato del backend

El módulo `src/mips/generator.ts` contiene la función pública `generateMips(tac)`.

El resultado usa cuatro estados: `not-requested`, `skipped`, `completed` y `failed`. Un ensamblador parcial nunca se presenta como listo.

Estado	Significado
not-requested	la fase no fue solicitada
skipped	no existe TAC válido por errores anteriores
completed	ensamblador generado y validado
failed	el backend detectó una inconsistencia

5.2. Proceso de generación

El backend realiza el trabajo en etapas deliberadamente separadas:

1. **Indexación.** Registra frames, funciones, layouts, vtables, constantes y símbolos globales.
2. **Emisión de datos.** Declara strings internados, constantes flotantes, vtables, descriptores estáticos y almacenamiento global.
3. **Emisión de texto.** Traduce el cuerpo global como `main` y cada función bajo una etiqueta ùnica.
4. **Inclusión de runtime.** Agrega solamente las rutinas auxiliares requeridas por las operaciones presentes.
5. **Validación.** Comprueba que referencias de salto, llamadas y direcciones correspondan a etiquetas definidas.
6. **Mirilla.** Elimina patrones redundantes y conserva métricas antes/después.

Separar estas etapas permite resolver referencias adelantadas: una función puede llamar a otra declarada después porque las etiquetas se conocen antes de emitir los cuerpos. También hace posible generar vtables completas antes de construir instancias.

5.3. Resolución de almacenamiento

Los operandos globales se convierten en etiquetas del segmento `.data`. Los operandos de la función activa usan su offset relativo a `$fp`. Las constantes se materializan con `li`, `la` o `l.s`. Los valores

persistentes se almacenan después de cada cuádruplo; así no dependen de que un registro temporal sobreviva a saltos o llamadas.

Clase de valor	Ubicación	Motivo
Global	etiqueta en `.data`	vida durante todo el programa
Parámetro	offset de `$fp`	acceso estable dentro de la invocación
Local	offset de `$fp`	aislamiento entre llamadas y recursión
Temporal TAC	slot del frame	persistencia entre cuádruplos
Constante entera	inmediato con `li`	no necesita memoria propia
String/float constante	pool en `.data`	formato binario y reutilización
Objeto compuesto	puntero a heap	tamaño dinámico y vida extendida
Captura	celda compartida en heap	sobrevivir al retorno del propietario

El backend no vuelve a buscar una variable local por nombre cuando existe `symbolId`. De esta forma, una declaración interior y otra exterior con el mismo texto ocupan slots diferentes. La regresión correspondiente imprime `2` dentro del bloque y `1` después de salir.

5.4. Asignador getReg

El asignador administra `$t0` a `$t7` y reinicia su conjunto de uso en cada instrucción TAC. Esta decisión corresponde a la naturaleza de tres direcciones: los valores vivos entre instrucciones están en slots, mientras que los registros sirven para cargar operandos y producir un resultado local.

Los registros `$t8` y `$t9` se reservan para soporte interno, como punteros de función y comparaciones. La estrategia reporta asignaciones totales, máximo simultáneo y spills. Si una instrucción agotara el conjunto disponible, el backend produciría un diagnóstico en lugar de reutilizar silenciosamente un registro vivo.

5.5. Selección de instrucciones

Familia TAC	MIPS	Observación
MOV	li/la/lw y sw	depende de la clase del operando
ADD/SUB/MUL	addu/subu/mul	enteros de 32 bits
DIV/MOD	div con mflo/mfhi	cociente o residuo
EQ–GE	seq, sne, slt, sle, sgt, sge	resultado booleano 0/1
IF/GOTO	beq, bne, j	delay slot explícito
CALL	jal o jalr	nombre directo o puntero
RETURN	$v0/$f0	salta a un epílogo común
PRINT	syscall	seleccionado por tipo

5.6. Convención de llamadas paso a paso

Para una llamada `f(a,b)`, el llamador evalúa y apila los argumentos de izquierda a derecha. Luego ejecuta `jal` para una función directa o `jalr` para un descriptor de método/closure. Al entrar, el callee reserva un frame alineado, guarda `$s7`, `$fp` y `$ra`, establece el nuevo frame pointer y copia los argumentos a slots conocidos.

El valor de retorno entero, booleano, string o referencia se coloca en `$v0`; un float usa `$f0`. Todos los retornos saltan a un epílogo común, que restaura el entorno del closure, la dirección de retorno y el frame anterior. Finalmente, el llamador retira exactamente los argumentos pertenecientes a esa llamada.

La limpieza exacta es indispensable para llamadas anidadas. Cuando `sumar` recibe los resultados de dos llamadas a `multiplicar`, el primer argumento ya calculado debe permanecer disponible mientras se ejecuta la segunda llamada interna. Limpiar toda la zona de argumentos produciría un resultado incorrecto aunque cada función aislada pareciera funcionar.

Los métodos reciben el objeto receptor como primer parámetro implícito. Los constructores siguen la misma convención, pero retornan el receptor inicializado. En herencia, el constructor hijo invoca primero al constructor padre compatible y después inicializa sus propios campos.

5.7. Arreglos y matrices

Un arreglo ocupa una palabra de longitud seguida por sus elementos. Reservar un arreglo de n elementos requiere $(n+1) \times 4$ bytes. Para acceder al elemento i, el runtime verifica que la referencia no sea nula, que $i \geq 0$ y que $i < n$; solo entonces calcula $base + 4(i+1)$.

Una matriz no requiere un formato adicional: es un arreglo cuyos elementos son punteros a otros arreglos. Por ello `matriz[f][c]` se traduce como dos accesos seguros consecutivos. Esta composición conserva longitudes diferentes por fila y reutiliza las mismas operaciones TAC y MIPS.

5.8. Objetos, layouts y despacho dinámico

Los objetos reservan `ClassLayout.instanceSize`; el primer word apunta a la vtable de la clase real y los campos usan offsets calculados en TAC. Un layout hijo incorpora los campos heredados antes de agregar los propios, de modo que un método base mantiene offsets válidos cuando recibe una instancia hija.

Cada método ocupa un slot estable. Si una clase hija sobrescribe un método, conserva el slot y reemplaza la dirección en su vtable. Una llamada mediante una referencia base carga la vtable del objeto, selecciona el slot y ejecuta `jalr`; por eso se invoca el método de la clase real, no el que sugiere solamente el tipo estático.

Antes de leer o escribir un campo se comprueba que el receptor no sea `null`. Si existe un handler activo, el error se transfiere al `catch`; de lo contrario, el runtime imprime el mensaje y termina de forma controlada, sin provocar una excepción interna de MARS.

5.9. Closures y capturas

Un valor función se representa mediante un descriptor. El primer word contiene la dirección del código; los words siguientes contienen punteros a celdas capturadas. Una función sin capturas puede usar un descriptor estático, mientras un closure con entorno necesita un descriptor en heap.

Al invocar el descriptor, `$s7` apunta al entorno activo. `LOAD_CAPTURE` y `STORE_CAPTURE` acceden a la celda compartida, no a una copia. Por eso dos llamadas al contador creado por una función anidada observan y modifican el mismo estado aun después de que la función creadora retornó.

5.10. Strings y flotantes

Las cadenas constantes se internan para evitar duplicados. La concatenación solicita memoria y copia bytes; la igualdad compara byte a byte. Los flotantes usan instrucciones `l.s`, `s.s`, `add.s`, `sub.s`, `mul.s` y `div.s`. La promoción entera se realiza con `mtc1` y `cvt.s.w`.

Los booleanos se imprimen como `true` o `false`, y `null` como texto. Esta presentación mantiene la semántica del lenguaje fuente y evita exponer al usuario valores internos como 0, 1 o una dirección.

5.11. Errores de runtime y try/catch

El backend protege cuatro familias de operaciones: referencia de arreglo nula, ìndice fuera de límites, referencia de objeto nula y división/módulo entre cero. Para floats, la comparación con cero ocurre en el coprocesador antes de ejecutar `div.s`.

`TRY_BEGIN` instala un registro dinámico que conserva el handler anterior, `$sp`, `$fp`, `$s7` y la etiqueta del bloque de captura. Si ocurre un error, el runtime restaura ese contexto antes de saltar al `catch`. Esta restauración permite capturar un fallo originado dentro de una llamada anidada sin continuar con un frame corrupto.

`TRY_END` desmonta el registro cuando la región termina normalmente. Los saltos generados para `break`, `continue` o `return` también respetan los handlers que abandonan, evitando que un registro obsoleto capture un error posterior.

5.12. Mirilla

La optimización local elimina movimientos de un registro hacia sí mismo, ajustes nulos de pila, saltos hacia la etiqueta siguiente y `nop` consecutivos. Se conserva la salida previa para auditoría y se informa cuántas instrucciones se removieron.

Las reglas son deliberadamente conservadoras: operan sobre patrones cuya equivalencia no depende de información global. No se reordenan accesos a memoria, llamadas ni ramas condicionales. Esto satisface el requisito de mirilla sin introducir una fase de optimización que pudiera modificar el comportamiento observable.

5.13. Validación del ensamblador generado

Antes de devolver `completed`, el backend inspecciona las referencias producidas por saltos, ramas, `jal` y carga de direcciones. Una etiqueta ausente se reporta como diagnóstico y bloquea la exportación. Esta validación estructural complementa, pero no reemplaza, la ejecución real en MARS: una salida puede tener etiquetas completas y aun contener un error de convención de llamadas, motivo por el cual existen ambos niveles de prueba.

La validación final comprueba, como mínimo, las siguientes propiedades:

Propiedad	Razón
Entrada definida	el segmento de texto debe contener un punto de inicio ejecutable
Etiquetas ùnicas	dos bloques no pueden compartir el mismo destino simbólico
Referencias resueltas	todo salto, rama, llamada y carga de dirección debe apuntar a una etiqueta existente
Frames conocidos	cada operando local debe resolverse contra el registro de activación correcto
Runtime consistente	las rutinas auxiliares se emiten solo si son referidas y una sola vez
Resultado sin diagnósticos	cualquier inconsistencia cambia el estado a `failed` y deshabilita la descarga

Esta etapa convierte errores del propio compilador en mensajes controlados. Sin ella, una referencia sin resolver podría descubrirse recién al ensamblar o, peor aún, una dirección incorrecta podría producir un fallo durante la ejecución sin indicar qué decisión del backend lo originó.

CAPÍTULO 6

Ejemplo completo de traducción

6.1. Programa fuente

El siguiente fragmento combina una función, una llamada y una expresión aritmética. Es pequeño, pero permite observar todas las fronteras relevantes entre representaciones.

```
function doble(n: integer): integer {
  return n * 2;
}

let base: integer = 5;
let resultado: integer = doble(base + 1);
print(resultado);
```

El lexer reconoce palabras reservadas, identificadores, literales, signos y operadores. El parser comprueba que la declaración, los tipos, el bloque y las llamadas coincidan con la gramática. La semántica registra `doble`, verifica que recibe un entero, infiere que `base+1` es entero y confirma que el retorno puede asignarse a `resultado`.

6.2. Representación intermedia

Una versión simplificada del TAC producido es:

```
FUNC_BEGIN doble
PARAM_DEF n
t0 = n * 2
RETURN t0
FUNC_END doble

base = 5
t1 = base + 1
PARAM t1
t2 = CALL doble, 1
resultado = t2
PRINT resultado
```

La expresión anidada se vuelve lineal. El valor `base+1` se calcula antes de la llamada; la cantidad `1` asociada a `CALL` indica cuántos argumentos debe retirar el llamador. El temporal de retorno se copia al símbolo destino y luego se imprime.

6.3. Decisiones de almacenamiento

El frame de `doble` reserva slots para el parámetro `n` y el temporal `t0`, además de las palabras de control. El cuerpo global posee almacenamiento para `base`, `t1`, `t2` y `resultado`. La identidad de cada símbolo determina el slot; el texto del nombre solo se conserva para lectura humana.

6.4. Esqueleto MIPS resultante

El ensamblador real agrega etiquetas y offsets concretos. El siguiente extracto conceptual muestra la relación con TAC:

```
fn_doble:
  addiu $sp, $sp, -FRAME_SIZE
  sw    $ra, RA_OFFSET($sp)
  sw    $fp, FP_OFFSET($sp)
  move  $fp, $sp
  # cargar n, multiplicar por 2 y colocar retorno
  lw    $t0, N_OFFSET($fp)
  li    $t1, 2
  mul   $t2, $t0, $t1
  move  $v0, $t2
  j     fn_doble_epilogue
  nop

main:
  # base = 5; calcular base + 1
  li    $t0, 5
  addiu $t1, $t0, 1
  addiu $sp, $sp, -4
  sw    $t1, 0($sp)
  jal   fn_doble
  nop
  addiu $sp, $sp, 4
  # imprimir el retorno
  move  $a0, $v0
  li    $v0, 1
  syscall
```

El extracto omite detalles generados, pero hace visibles las obligaciones: reservar/restaurar el frame, preservar la dirección de retorno, pasar el argumento, recibir el resultado y seleccionar syscall 1 por tratarse de un entero. La salida esperada en MARS es `12`.

6.5. Qué ocurre si el programa es inválido

Si `doble` fuera invocada con un string, la semántica emitiría `SEM007`. No se construiría TAC para el programa y el ejemplo MIPS anterior nunca se produciría. Esta conducta es una propiedad del pipeline,

no una decisión visual de ocultar texto: el backend no recibe un `TacGenerationResult` válido.

6.6. Lectura comparativa del ejemplo

El mismo comportamiento puede observarse en tres niveles. Cada nivel responde una pregunta diferente y juntos permiten explicar el proceso sin confundir abstracciones.

Nivel	Pregunta	Evidencia
Compiscript	¿Qué desea expresar el programador?	tipos, función, llamada y print
TAC	¿En qué orden ocurren las operaciones?	temporales, PARAM, CALL y RETURN
MIPS	¿Cómo se ejecuta en la máquina?	frame, pila, registros, jal y syscall

La salida `12` demuestra el resultado observable, mientras la inspección de TAC y MIPS explica por qué se obtiene. En una evaluación conviene mostrar ambos aspectos: ejecutar el archivo comprueba compatibilidad; recorrer las representaciones demuestra que el ensamblador se originó en TAC y no en una ruta paralela.

CAPÍTULO 7

Interfaz y experiencia de uso

El flujo normal para Entrada válida y Archivo propio solicita modo `mips`; por tanto, un programa correcto llega automáticamente al backend. El modo TAC permanece disponible como demostración aislada.

El pipeline visual usa el estado real de cada fase. MIPS aparece como omitido ante errores previos, fallido ante diagnósticos del backend y listo para MARS solo en `completed`. El panel muestra estado, instrucciones finales, optimizaciones, diagnósticos, cantidad TAC y pico de registros.

El selector de archivos está dentro de la GUI y acepta `.cps`. Los resultados se muestran en la misma interfaz; la consola no es necesaria para la evaluación funcional.

La interfaz renovada conserva el flujo del compilador y reorganiza su presentación como un workbench: barra superior unificada, controles compactos, editor central, explorador lateral e inspector vertical. El sistema visual emplea superficies translúcidas, radios consistentes y acentos violeta, cian y verde. El tema claro mantiene la misma jerarquía mediante fondos azulados suaves, contraste oscuro y paneles blancos translúcidos, sin perder legibilidad ni estados semánticos.

7.1. Distribución del workbench

Zona	Función
Barra superior	archivo, modo, ejecución, tema y acciones principales
Explorador lateral	navegación entre entrada, fases y vistas disponibles
Editor central	edición Monaco con numeración, selección y resaltado
Pipeline	estado observable de lexer, parser, semántica, TAC y MIPS
Inspector derecho	tokens, àrboles, símbolos, frames, TAC y ensamblador
Panel de problemas	diagnósticos filtrables con severidad y ubicación
Barra de estado	resumen de entrada, fase activa y resultado de compilación

La jerarquía visual separa acciones de contenido: compilar y cargar permanecen visibles, mientras los detalles técnicos se agrupan en pestañas e inspectores. Esta organización reduce desplazamientos y permite demostrar el pipeline sin abrir herramientas externas para cada fase.

7.2. Flujo de uso

1. El usuario escribe Compiscript o selecciona un archivo `.cps` desde Archivo.
2. Al presionar Analizar, el orquestador ejecuta las fases permitidas por el modo.
3. El pipeline actualiza cada estado y el panel de problemas presenta diagnósticos acumulados.

4. Para una entrada válida, el inspector habilita TAC, frames, MIPS y métricas.
5. La descarga `compiscript.asm` solo aparece cuando MIPS está completo.
6. El usuario ejecuta el archivo con MARS y compara la salida con el resultado esperado.

7.3. Exportación de evidencia

Además del ensamblador, la aplicación puede exportar resultados de análisis, tokens, símbolos y TAC en formatos legibles. Estas exportaciones son ùtiles para explicar una calificación, reproducir un fallo o comparar dos versiones. La descarga conserva los diagnósticos y estados omitidos; no presenta una entrada rechazada como si hubiera llegado al final del pipeline.

CAPÍTULO 8

CLI y ejecución externa

La CLI comparte `analyzeInput`; no posee un compilador alterno. El modo MIPS acepta `-out`:

```
npm run cli -- examples/mips/07_recursion.cps \
  --mode mips --out factorial.asm
java -jar "C:\Herramientas\Mars4_5.jar" nc ae1 factorial.asm
```

El script `scripts/run-mars.mjs` busca el JAR en el segundo argumento, en `MARS_JAR` o en `tools/Mars4\_5.jar`. En el equipo de entrega se encuentra en `C:/Herramientas/Mars4\_5.jar`; puede seleccionarse sin modificar el repositorio mediante la variable de entorno o como segundo argumento del runner. MARS sigue siendo una herramienta externa: el IDE genera y exporta, pero no simula internamente.

8.1. Modos de la CLI

Modo	Fase final	Uso principal
lexer	tokens	aislar reconocimiento y errores léxicos
parser	CST	comprobar gramática y recuperación
semantic	modelo tipado	revisar diagnósticos, símbolos y àmbitos
tac	representación intermedia	inspeccionar cuádruplos, CFG y frames
mips	ensamblador	generar `.asm` para MARS

La CLI devuelve un código distinto de cero si una fase solicitada falla y no escribe el archivo de salida cuando el programa es inválido. Esta propiedad permite integrarla a scripts y CI sin depender de interpretar texto de consola.

8.2. Ejecución reproducible en PowerShell

```
npm.cmd ci
npm.cmd run cli -- .\examples\mips\07_recursion.cps `
  --mode mips --out .\factorial.asm
$env:MARS_JAR = "C:\Herramientas\Mars4_5.jar"
npm.cmd run mars -- .\factorial.asm
```

La salida esperada es `120`. La opción `ae1` utilizada por el runner hace que MARS retorne código 1 si el ensamblado falla; de este modo un archivo sintácticamente incorrecto no puede aparecer como prueba aprobada.

8.3. Diagnóstico de la ejecución externa

Situación	Comprobación recomendada
Java no se reconoce	ejecutar `java -version` y confirmar Java 17 o posterior
MARS no se encuentra	verificar `C:/Herramientas/Mars4\_5.jar` o definir `MARS_JAR`
El ensamblado falla	usar el runner con `ae1` y revisar línea/etiqueta informada por MARS
No aparece salida	confirmar que el programa alcanza `PRINT` y que la syscall corresponde al tipo
La suite omite MARS	definir `REQUIRE_MARS=1` para convertir la omisión en un error explícito
Salida inesperada	comparar primero TAC, frames y cantidad de argumentos de cada `CALL`

La ruta del JAR es configuración del equipo, no una dependencia versionada. Mantenerla fuera del repositorio evita distribuir software externo y permite que cada integrante o runner seleccione su propia ubicación.

CAPÍTULO 9

Pruebas y resultados

9.1. Estrategia de verificación

La verificación se organiza como una pirámide. Las pruebas unitarias detectan errores locales con rapidez; las de integración recorren varias fases; las regresiones preservan correcciones de auditoría; y las ejecuciones MARS validan el comportamiento observable del programa ensamblado.

Nivel	Qué comprueba	Ejemplo de fallo detectado
Lexer/parser	tokens, mensajes y recuperación	detenerse tras el primer error
Semántica	tipos, àmbitos, herencia y flujo	aceptar un argumento incompatible
TAC	orden, destinos, frames y metadatos	salto a bloque incorrecto
Backend estructural	opcodes, etiquetas, runtime y mirilla	omitir una vtable o un handler
MARS	salida real y errores de runtime	convención de pila incorrecta
UI/exportación	estados, controles y archivos	habilitar descarga tras un error
Build/auditoría	tipado, empaquetado y dependencias	importar un módulo inválido

Cada corrección de alto impacto incorpora una regresión. Por ejemplo, el despacho dinámico se verifica con una referencia base que contiene una instancia hija; el *shadowing* imprime los valores interior y exterior; y la limpieza de argumentos se ejercita con llamadas anidadas. Esta práctica evita que una mejora posterior reintroduzca un defecto ya resuelto.

9.2. Suite automatizada

La suite final contiene 232 casos en 15 archivos: 191 se ejecutan sin herramientas externas y 41 validan la salida real en MARS cuando Java y el JAR están disponibles. Las pruebas del backend verifican generación completa, convención de pila, arreglos, runtime de strings, llamadas anidadas, closures, despacho dinámico, constructores heredados, errores de objeto nulo y división entre cero, recuperación de `try/catch`, bloqueo por errores y mirilla. Las pruebas heredadas continúan verificando lexer, parser, semántica, símbolos, TAC, CFG, UI y la exportación del modo MIPS.

Los 191 casos independientes de herramientas externas permiten obtener retroalimentación incluso si MARS no está instalado. Sin embargo, no se confunden con ejecución de ensamblador: cuando

`REQUIRE_MARS=1`, la ausencia de Java o del JAR hace fallar la suite en lugar de omitir silenciosamente los 41 escenarios.

9.3. Validación real en MARS

La suite de integración contiene 41 escenarios MARS, incluidos resultados positivos, errores de arreglos y objetos, división entre cero, despacho dinámico, constructores heredados y capturas mediante `try/catch`. El workflow instala Java 17, prepara MARS 4.5 y exige todos los escenarios. Algunos resultados representativos son:

Caso	Salida
aritmética	14
funciones y pila	42
arreglos y foreach	6
strings y booleanos	Hola MIPS; true
flotantes	4.0
factorial recursivo	120
closure	1
clase, método y llamada anidada	2; 6; 12
shadowing de bloque	2; 1
despacho dinámico	Toby ladra
división entre cero capturada	mensaje de runtime y continuación

Las matrices se validan además con un caso manual reproducible que crea `integer[][]`, modifica una celda y comprueba las salidas `9` y `3`. Se documenta por separado para no atribuir a la suite permanente una cobertura que pertenece a la guía de aceptación.

El caso de errores mezclados produce diagnósticos, deja TAC/MIPS en `skipped` y no crea un archivo ensamblador.

9.4. Pruebas negativas y recuperación

Los casos negativos no buscan solamente confirmar que "hay un error". Se espera una cantidad y categoría razonables de diagnósticos, que el analizador alcance construcciones posteriores, que no repita indefinidamente el mismo mensaje y que bloquee las fases incompatibles.

Entrada	Resultado esperado	Propiedad observada
Tres caracteres inválidos	tres errores léxicos	continuación y deduplicación
Tres puntos y coma ausentes	tres errores sintácticos	sincronización del parser
Errores combinados	léxicos y sintácticos juntos	fases independientes

Entrada	Resultado esperado	Propiedad observada
Errores semánticos variados	códigos SEM distintos	acumulación del visitor
Arreglo covariante mutable	SEM003	seguridad de tipos
300 paréntesis	diagnóstico estructural ùnico	protección contra desbordamiento

Los errores de runtime tienen una expectativa diferente. La compilación sí es válida, por lo que TAC y MIPS se generan; la comprobación ocurre durante la ejecución. Un fallo no capturado imprime un mensaje controlado y termina antes de ejecutar instrucciones posteriores. Dentro de `try/catch`, el mensaje se entrega a la variable de captura y el programa continúa.

9.5. Comandos de verificación

```
npm ci
npm run check
npm test
npm run build
npm run cli:mips
npm audit
```

TypeScript, 191 pruebas independientes de herramientas externas y el build terminan correctamente; los 41 escenarios MARS también fueron ejecutados y son obligatorios en CI. La auditoría completa de dependencias reporta cero vulnerabilidades.

9.6. Interpretación de resultados

La cantidad de pruebas por sí sola no demuestra corrección; su valor proviene de la diversidad de propiedades observadas. Las pruebas de estructura confirman que se emiten instrucciones esperadas, mientras MARS confirma que la combinación completa produce una salida. Las pruebas negativas demuestran el bloqueo y la recuperación, y el build confirma que los contratos TypeScript permanecen coherentes.

No se reemplaza MARS con una expresión regular ni con un intérprete propio. Los procesos de prueba escriben ensamblador temporal, invocan el JAR oficial en modo CLI y comparan stdout/código de salida. Esta distinción es importante porque errores de offsets, frames o delay slots pueden pasar una inspección textual y fallar solamente al ejecutar.

9.7. Integración continua

El workflow de GitHub instala Node y Java 17, restaura dependencias mediante el lockfile, prepara MARS con verificación de hash y activa `REQUIRE_MARS=1`. Luego ejecuta check, pruebas, build y

ejemplos TAC/MIPS. Los eventos de pull request, cambios a `main` y releases utilizan esta verificación antes del empaquetado.

Para distribución, Windows produce instalador NSIS y ejecutable portable; macOS y Linux requieren runners de sus respectivos sistemas. La firma de binarios y la publicación de un tag permanecen como acciones externas al código fuente.

CAPÍTULO 10

Trazabilidad con el enunciado

Requisito	Estado	Evidencia
ANTLR obligatorio	Cumple	gramática y código generado
MIPS desde TAC	Cumple	API `generateMips(tac)`
GUI y selector de archivo	Cumple	React/Monaco y menú Archivo
Resultados dentro de GUI	Cumple	paneles de fases, TAC y MIPS
Múltiples errores y recuperación	Cumple	listeners, estrategia ANTLR y visitors
No TAC/MIPS ante errores	Cumple	estados y pruebas negativas
Mirilla	Cumple	optimizador y métricas
Ensamblador ejecutable	Cumple	41 escenarios MARS aprobados y exigidos en CI
Casos de prueba propios	Cumple	ejemplos MIPS, TAC, semánticos y de rúbrica
Evitar cascadas y ciclos	Cumple	normalización, límites y regresiones de recuperación
Arquitectura destino documentada	Cumple	MIPS32, MARS, frames, heap y syscalls
No código objeto ni ejecución interna	Cumple	exportación a herramienta externa

10.1. Evidencia observable durante la demostración

La trazabilidad no depende de afirmar que un archivo existe. Para una demostración completa se carga `examples/tac/13\_programa\_integral.cps`, se ejecuta el pipeline y se observan dentro de la interfaz tokens, àrbol, símbolos, TAC, frames y MIPS. El ensamblador descargado debe imprimir `2`, `6` y `12` en MARS.

Luego se carga `examples/mips/05\_bloqueo\_por\_errores.cps`. La interfaz debe mostrar los problemas, marcar las fases posteriores como omitidas y deshabilitar la descarga. Esta segunda mitad es tan importante como el caso positivo, porque demuestra el requisito explícito de no generar representación intermedia ni ensamblador ante errores previos.

10.2. Responsabilidad individual

El grupo individual autorizado **Daniel-Barillas** corresponde a **Pablo Daniel Barillas Moreno, carné 22193**; la misma identidad aparece en la portada y los metadatos.

CAPÍTULO 11

Limitaciones y trabajo futuro

Compiscript no expone una sentencia `throw` de usuario; sin embargo, el backend utiliza `TRY_BEGIN`, `TRY_END` y `CATCH_BEGIN` para registrar handlers dinámicos. Los errores de runtime definidos restauran `$sp`, `$fp` y `$s7`, incluso al atravesar una llamada, y transfieren el mensaje al bloque `catch`.

La memoria de heap se obtiene mediante syscall 9 y no se libera, porque MARS no ofrece un recolector de basura integrado y el alcance no exige administración manual completa. Esta decisión es adecuada para programas educativos de duración limitada, pero no para procesos de larga vida con asignación intensiva.

No existe una fase global de optimización, de acuerdo con el alcance: se aplica mirilla local. Los valores persistentes regresan a slots entre instrucciones, una estrategia robusta y explicable que sacrifica parte del rendimiento frente a una asignación global basada en liveness.

MARS implementa un entorno educativo y syscalls particulares; el ensamblador generado está orientado a esa compatibilidad. Portarlo a Linux MIPS o a hardware real requeriría revisar ABI, syscalls, formato ejecutable y enlazado.

Como evolución podrían incorporarse una sentencia `throw` en el lenguaje, tipos de excepción, recolección de memoria, asignación global de registros por liveness, optimizaciones entre bloques y un backend alternativo. Estas mejoras no son necesarias para el alcance actual y deberían introducirse con nuevas pruebas de equivalencia semántica.

CAPÍTULO 12

Conclusiones

El Proyecto 3 integra de forma coherente las fases anteriores y produce ensamblador MIPS32 real desde TAC. La convención de llamadas permite funciones, anidamiento y recursión; el modelo de heap cubre arreglos, objetos, cadenas y closures; los estados impiden exportar resultados inválidos; y la interfaz hace observable todo el proceso.

La evidencia combina pruebas unitarias, integración, build de producción y ejecución en MARS 4.5. Por ello, la afirmación de compatibilidad no depende solamente de documentación o de ejemplos de ensamblador, sino de resultados reproducibles.

Las correcciones de auditoría muestran además que los riesgos más delicados aparecen en los límites entre capas: identidad semántica y almacenamiento, argumentos y frames, tipo estático y clase real, o error de runtime y restauración de contexto. Mantener contratos explícitos y regresiones ejecutables resultó más valioso que concentrar toda la lógica en un generador monolítico.

Finalmente, la GUI no es un accesorio separado del compilador. Presenta los mismos estados y estructuras que usan la CLI y las pruebas, por lo que permite explicar cómo una entrada se transforma y por qué una entrada inválida se bloquea. El producto final cumple así una función técnica y una función pedagógica.

CAPÍTULO A

Mapa de archivos

Archivo	Responsabilidad
`src/mips/generator.ts`	backend, runtime, mirilla y diagnósticos
`src/tac/types.ts`	contrato de TAC, frames y layouts
`src/lib/analyze.ts`	orquestación y bloqueo entre fases
`src/ui/App.tsx`	flujo completo para entrada propia
`src/ui/components/RightDock.tsx`	inspección y descarga MIPS
`src/cli/run.ts`	modo MIPS y escritura `.asm`
`scripts/run-mars.mjs`	invocación externa de MARS
`src/__tests__/mips.test.ts`	regresiones del backend
`src/__tests__/mips.execution.test.ts`	41 ejecuciones reales en MARS
`examples/mips`	demostraciones y caso de bloqueo
`docs/GUIA_PRUEBAS_COMPLETA.md`	casos manuales y resultados esperados
`docs/MATRIZ_REQUISITOS.md`	trazabilidad detallada de la rúbrica

CAPÍTULO B

Catálogo resumido de diagnósticos

Código	Categoría	Ejemplo
SEM001	identificador no declarado	usar `total` fuera de su alcance
SEM002	redeclaración	declarar dos veces un nombre en el mismo àmbito
SEM003	asignación incompatible	asignar string a integer o modificar const
SEM004	operador inválido	aplicar módulo a operandos float
SEM005	condición no booleana	usar integer como condición de if
SEM006–007	llamada inválida	aridad o tipo de argumento incorrecto
SEM008–011	control inválido	retorno incompatible o break/continue fuera de contexto
SEM012–014	clases y miembros	miembro inexistente, this o constructor inválido
SEM015–017	arreglos	ìndice no entero, valor no indexable o elementos incompatibles
SEM018	código inalcanzable	instrucción posterior a un terminador
SEM019–021	declaración y flujo	parámetro duplicado, herencia o switch inválido
SEM023	uso antes de inicialización	leer un local antes de asignarle valor

El diagnóstico incluye severidad, mensaje, línea, columna y, cuando aplica, información relacionada con la declaración original. Los warnings se conservan para inspección; los errores determinan el bloqueo de las fases posteriores.

CAPÍTULO C

Comandos de reproducción

C.1. Preparación y desarrollo

```
npm.cmd ci
npm.cmd run generate
npm.cmd run check
npm.cmd run dev
```

La interfaz queda disponible en `http://localhost:3000`. Para el uso cotidiano, una vez instaladas las dependencias, basta con `npm.cmd run dev`.

C.2. Verificación completa

```
$env:MARS_JAR = "C:\Herramientas\Mars4_5.jar"
$env:REQUIRE_MARS = "1"
npm.cmd run verify
npm.cmd audit --audit-level=moderate
```

El resultado documentado es: 15 archivos de prueba, 232 pruebas aprobadas, 41 ejecuciones MARS incluidas, build de producción aprobado y cero vulnerabilidades reportadas.

C.3. Generación y ejecución manual

```
npm.cmd run cli -- .\examples\mips\07_recursion.cps `
  --mode mips --out .\factorial.asm
npm.cmd run mars -- .\factorial.asm `
  "C:\Herramientas\Mars4_5.jar"
```

La salida esperada es `120`. Si MARS devuelve código distinto de cero, el ensamblado o la ejecución deben considerarse fallidos aunque el archivo `.asm` exista.